Rapport d'exécution des commandes

Hardening MongoDB – Séance 3

Étudiant : Hasna Daoui

4 avril 2026

Contexte

Ce rapport contient uniquement l'exécution des commandes réalisées lors des activités pratiques. Chaque bloc est précédé de son objectif et suivi des preuves demandées (succès / échec).

1 Activité 1 – Instance insecure vs secure

1.1 1. Démarrage d'une instance SANS authentification

```
docker rm -f mongo_insecure 2>/dev/null
docker run -d --name mongo_insecure -p 27017:27017 mongo:7
```

Sortie :

```
745b662b30fb15cc05752cc63eb58f1eb9274ab4a1a33b9f46b42bc8a2a01db6
```

1.2 2. Connexion anonyme et insertion

```
mongosh "mongodb://localhost:27017"
test> use noseclab
switched to db noseclab
noseclab> db.security_logs.insertOne({
  timestamp: new Date(),
  source_ip: "10.0.0.99",
  event_type: "test_insecure",
  message: "Ce document a ete insere sans authentification"
})
```

Preuve (insertion OK) :

```
{
  acknowledged: true,
  insertedId: ObjectId('69c4f853d484cd3ca2d805db')
}
```

1.3 3. Lister les bases (succès – preuve d'accès administrateur)

```
db.adminCommand({ listDatabases: 1 })
```

Preuve :

```
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'noseclab', sizeOnDisk: Long('8192'), empty: false }
  ],
  totalSize: Long('102400'),
  ok: 1
}
```

1.4 4. Nettoyage de l'instance insecure

```
docker rm -f mongo_insecure
```

1.5 5. Démarrage d'une instance AVEC authentification

```
docker run -d --name mongo_secure1 \
  -p 127.0.0.1:27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=adminSoc \
  -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! \
  mongo:7
```

Sortie :

```
bd7ecdd74bdcaa16fc1dc4f7dd1af7c8fb511c7dbb596b6d0704ab24d159fa32
```

1.6 6. Tentative de connexion anonyme (doit échouer)

```
mongosh "mongodb://localhost:27017"
test> show dbs
```

Preuve d'échec (attendue) :

```
MongoServerError[Unauthorized]: Command listDatabases requires
   authentication
```

1.7 7. Connexion authentifiée (succès)

```
mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
admin> db.adminCommand({ listDatabases: 1 })
```

Preuve :

```
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('102400'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('188416'),
  ok: 1
}
```

2 Activité 2 – Authentification et authenticationDatabase

2.1 1. Refus d'accès anonyme (déjà prouvé ci-dessus)

2.2 2. Connexion admin et vérification des utilisateurs

```
mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
admin> use admin
admin> db.getUsers()
```

Preuve :

```
{
  users: [
    {
      _id: 'admin.adminSoc',
      userId: UUID('adcb9b0b-d4c6-4a0a-a2e9-0edf3efdfbaa'),
      user: 'adminSoc',
      db: 'admin',
      roles: [ { role: 'root', db: 'admin' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
```

2.3 3. Insertion de test avec admin

```
use soc_lite
db.security_logs.insertOne({
  timestamp: new Date(),
  source_ip: "192.168.1.50",
  event_type: "auth_success",
  message: "Connexion admin verifiee",
  severity: "info"
})
```

Preuve :

```
{
  acknowledged: true,
  insertedId: ObjectId('69c4ff5fc579759a9ed805db')
}
```

2.4 4. Mauvaise authenticationDatabase (doit échouer)

```
mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/
   soc_lite"
```

Preuve d'échec :

```
MongoServerError: Authentication failed.
```

2.5 5. Bonne authenticationDatabase (succès)

```
mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
```

Preuve : connexion établie sans erreur.

3 Activité 3 – RBAC : moindre privilège

3.1 1. Insertion des données de test (admin)

```
mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
use soc_lite

db.security_logs.insertMany([
  { timestamp: ISODate("2026-03-23T08:15:00Z"), source_ip: "
     10.0.1.45", event_type: "login_failed", severity: "warning",
     message: "Echec SSH root", hostname: "srv-web-01" },
  { timestamp: ISODate("2026-03-23T08:15:03Z"), source_ip: "
     10.0.1.45", event_type: "login_failed", severity: "warning",
     message: "Echec SSH admin", hostname: "srv-web-01" },
  { timestamp: ISODate("2026-03-23T08:16:30Z"), source_ip: "
     10.0.1.45", event_type: "login_success", severity: "critical",
      message: "Connexion apres multiples echecs", hostname: "srv-
     web-01" }
])

db.incidents.insertMany([
  { incident_id: "INC-2026-0042", title: "Brute-force SSH",
     severity: "high", status: "open", created_at: ISODate("
     2026-03-23T08:20:00Z"), assigned_to: "analyst_dupont",
     description: "Multiples tentatives puis succes" },
  { incident_id: "INC-2026-0041", title: "Scan de ports", severity:
      "medium", status: "investigating", created_at: ISODate("
     2026-03-22T14:05:00Z"), assigned_to: "analyst_martin",
     description: "Scan SYN 1-1024" }
])

db.users.insertMany([
  { username: "analyst_dupont", email: "dupont@entreprise.fr",
     department: "SOC", clearance_level: "confidential" },
  { username: "analyst_martin", email: "martin@entreprise.fr",
     department: "SOC", clearance_level: "confidential" }
])
```

3.2 2. Création du rôle personnalisé

```
use soc_lite
db.createRole({
  role: "incidentManager",
  privileges: [{ resource: { db: "soc_lite", collection: "incidents
     " }, actions: ["find","insert","update","remove"] }],
  roles: []
})
```

Preuve : `{ ok: 1 }`

3.3 3. Création des utilisateurs

```
db.createUser({ user: "soc_app", pwd: "AppP@ss2026!", roles: [{ role: "readWrite", db: "soc_lite" }] })
db.createUser({ user: "soc_analyst_ro", pwd: "AnalystR0@2026!", roles: [{ role: "read", db: "soc_lite" }] })
db.createUser({ user: "soc_manager", pwd: "Mgr$ecure2026!", roles: [{ role: "incidentManager", db: "soc_lite" }] })
```

Preuve :

```
{ ok: 1 }
{ ok: 1 }
{ ok: 1 }
```

3.4 4. Tests – soc_analyst_ro

```
mongosh "mongodb://soc_analyst_ro:AnalystR0@2026!@localhost:27017/soc_lite"

db.security_logs.find().limit(2)          #     autorise
db.incidents.find({ severity: "high" })   #     autorise
db.security_logs.insertOne({ event_type: "test_write" })  # refuse
db.incidents.deleteOne({ incident_id: "INC-2026-0042" })  # refuse
db.users.find()                           #     refuse
```

Preuve de refus (exemple) :

```
MongoServerError: not authorized on soc_lite to execute command insert
```

3.5 5. Tests – soc_manager

```
mongosh "mongodb://soc_manager:Mgr\$ecure2026!@localhost:27017/soc_lite"

db.incidents.find()                       #     autorise
db.incidents.updateOne({ incident_id: "INC-2026-0042" }, { $set: { status: "investigating" } })  #     autorise
db.security_logs.find().limit(1)          #     refuse
db.users.find()                           #     refuse
```

3.6 6. Tests – soc_app

```
mongosh "mongodb://soc_app:AppP@ss2026!@localhost:27017/soc_lite"

```

```
db.security_logs.insertOne({ timestamp: new Date(), source_ip: "10.0.2.100", event_type: "firewall_drop" })  #     autorise
db.incidents.find({ status: "open" })     #     autorise
db.users.find()                           #        autoris  (limitation role built-in)
```